

SIMATS ENGINEERING
ASSESSMENT TOOL 3 – CASE STUDY

COURSE CODE: ITA1407

COURSE NAME: Ethical Hacking COURSE

OUTCOME COVERED

***CO4:** Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux. (BT4, BT5)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 Total Marks:100

Code of Conduct:

I __K Rishika_192525114 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: kavya

Question 1 — E-Commerce Apache Server Compromise (20 Marks)

1.1 Analysis of Possible Attack Vectors

The case describes three converging weaknesses that together explain how unauthorized transactions occurred: an outdated Apache installation with known CVEs, absent input validation on web forms, and the complete absence of a Web Application Firewall (WAF). Each of these maps to a distinct attack vector.

- **Outdated Apache / Linux stack:** Unpatched Apache versions frequently carry publicly disclosed CVEs (e.g., `mod_cgi`, `mod_proxy`, or HTTP request-smuggling flaws) that allow remote code execution, information disclosure, or denial of service. An attacker who fingerprints the server version can search public exploit databases and weaponize a known vulnerability directly.
- **Hidden directory access attempts:** Log entries showing enumeration of non-linked paths indicate directory/file brute-forcing (forced browsing), often used to locate admin panels, backup files, configuration files (`.env`, `.git`, `wp-config.php`-style files), or old/staging versions of the application that were never de-indexed.
- **Injection into input fields:** The presence of malicious scripts in input fields points to Cross-Site Scripting (XSS) and/or SQL Injection, both symptomatic of missing server-side input validation and output encoding. In an e-commerce context, this is exactly how attackers manipulate session cookies, hijack accounts, or alter transaction parameters.
- **Absence of a WAF:** With no WAF in place, none of the above attack traffic (SQL meta-characters, script tags, path traversal sequences) is filtered before reaching the application, so exploitation attempts that would otherwise be blocked at the edge succeed against the origin server.

1.2 Kali Linux Methodology

Phase 1 — Reconnaissance

Reconnaissance establishes the attack surface without touching the target intrusively.

- Passive OSINT: whois, theHarvester, and Shodan/Censys lookups to gather domain registration data, exposed subdomains, and any already-indexed sensitive files.
- Active fingerprinting: WhatWeb and Wappalyzer-style scans to identify the exact Apache version, OS, and any CMS/plugins in use.
- Content discovery: Gobuster or dirb/dirbuster against the web root to enumerate hidden directories, backup files, and admin interfaces referenced in the incident logs.
- Subdomain enumeration: Sublist3r or amass to reveal staging or forgotten subdomains that may run the same vulnerable stack.

Phase 2 — Vulnerability Scanning

- Nmap with the -sV and --script=vuln (NSE) flags to correlate open ports/services with known CVEs.
- Nikto to scan specifically for outdated server banners, dangerous default files, misconfigurations, and missing security headers.
- OpenVAS/Greenbone (or Nessus if licensed) for an authenticated/unauthenticated vulnerability sweep of the host.
- searchsploit to cross-reference the fingerprinted Apache version and any identified CMS/plugin versions against the Exploit-DB offline mirror.

Phase 3 — Exploitation (authorized test only)

- Where a matching CVE exists, validate it in a controlled manner using the corresponding Metasploit module or a proof-of-concept script, working strictly within the scope of an authorized penetration test.
- Use Burp Suite to manually probe the flagged input fields for reflected/stored XSS and injection flaws identified during scanning, confirming that unsanitized input reaches the response or the database layer.
- Document each confirmed finding with evidence (request/response pairs, screenshots) rather than escalating further than necessary to prove impact — this is the standard 'prove, don't destroy' principle of professional penetration testing.

1.3 Layered Defense Strategy

Server Hardening

- Patch Apache and the underlying OS on a defined cadence; subscribe to CVE/security advisories for the exact stack in use.
- Disable directory listing (Options -Indexes), remove default/sample pages, and turn off unused modules (mod_status, mod_info, mod_cgi if unneeded).
- Run the web server as a low-privilege user and apply filesystem permissions restricting access to configuration and data files.
- Enforce HTTPS/TLS with modern cipher suites and HSTS.

Secure Coding Practices

- Use parameterized queries / prepared statements or an ORM to eliminate SQL Injection.
- Apply context-aware output encoding and a strict Content Security Policy (CSP) to neutralize XSS.
- Validate all input server-side using allow-lists rather than relying on client-side checks.
- Return generic error messages to users while logging detailed errors server-side only.

Network-Level Protections

- Deploy a WAF (e.g., ModSecurity with the OWASP Core Rule Set) in front of the application to filter malicious payloads.
- Segment the e-commerce application and database tiers, restrict database access to the application server only, and enforce least privilege on service accounts.
- Add an IDS/IPS and centralized logging/SIEM to detect and alert on reconnaissance patterns (directory brute-forcing, repeated 4xx/5xx codes) before they escalate.
- Schedule periodic vulnerability assessments and penetration tests to catch regressions.

```
ip a
ip route
whois <target-domain>
dig <target-domain>
nslookup <target-domain>
ping -c 4 <target-ip>
curl -I http://<target-ip>
whatweb http://<target-ip>
nmap -sV -sC <target-ip>
nmap -p- <target-ip>
nmap -O <target-ip>
nikto -h http://<target-ip>
gobuster dir -u http://<target-ip> -w /usr/share/wordlists/dirb/common.txt
nmap --script vuln <target-ip>
```

Question 2 — SQL Injection & XSS in a Financial Web Application (20 Marks)

2.1 How the Vulnerabilities Are Exploited

SQL Injection (SQLi) occurs because user-supplied input (login fields, search boxes, account parameters) is concatenated directly into SQL statements instead of being parameterized. Because the application also returns verbose database error messages, an attacker can use error-based or union-based techniques to progressively learn the database schema (table names, column names) and ultimately extract customer records such as account numbers and balances.

Cross-Site Scripting arises from the same root cause applied to HTML/JavaScript context: user input is rendered back into pages without encoding. Stored XSS is especially dangerous in a financial portal because a script injected once (e.g., in a profile field or transaction note) executes for every user — including other customers or administrators — who later views that page, enabling session-cookie theft, unauthorized actions performed in the victim's session, or credential harvesting via a fake login overlay.

2.2 Step-by-Step Methodology with Burp Suite and SQLmap

- Step 1 — Map the application: Browse the application with Burp Suite's proxy intercepting traffic, building a complete site map of every parameter, form field, cookie, and API endpoint that accepts input.
- Step 2 — Identify injection points: Use Burp Repeater to manually alter individual parameters and observe changes in the response (error messages, timing differences, altered result sets) that suggest the input reaches a query unsanitized.
- Step 3 — Automated fuzzing: Use Burp Intruder to systematically test each candidate parameter with a payload set, and use the Burp Scanner (or OWASP ZAP as an alternative) to flag likely SQLi and XSS sink points across the whole site map.
- Step 4 — Confirm and characterize SQLi with SQLmap: Export the flagged request from Burp and feed it to SQLmap to confirm the injection type (boolean-blind, error-based, time-based, or UNION-based), enumerate available databases and tables (--dbs, --tables), and — strictly within the authorized test scope — retrieve a sample record to prove data exposure without exfiltrating the full dataset.
- Step 5 — Confirm XSS: Insert non-destructive marker payloads into each candidate field and check whether they execute unescaped in the rendered page (reflected) or persist and execute for other sessions (stored), using Burp's rendered response view to verify execution context.
- Step 6 — Assess session handling: Inspect cookies in Burp for the Secure, HttpOnly, and SameSite attributes, check whether session identifiers are regenerated after login, and test for session fixation or predictable token patterns.

2.3 Impact on the CIA Triad

- **Confidentiality:** SQL Injection allows direct extraction of sensitive customer data (PII, account/transaction records), representing a severe confidentiality breach with regulatory (e.g., PCI-DSS, data-protection law) consequences.
- **Integrity:** Beyond read access, injection can be used to modify or delete records (e.g., altering balances or transaction logs), and stored XSS can be used to silently perform actions on behalf of a logged-in victim, undermining the integrity of both data and user sessions.
- **Availability:** Poorly-formed or resource-intensive injected queries can degrade database performance or crash services; stored XSS payloads that redirect or lock out users also effectively deny them normal access to the application.
-

```
iwconfig
ip link
iw dev
sudo airmon-ng
sudo airmon-ng start wlan0
sudo airodump-ng wlan0mon
sudo airodump-ng --bssid <LAB-AP-MAC> -c <CHANNEL> -w lab_capture wlan0mon
wireshark lab_capture-01.cap
sudo airmon-ng stop wlan0mon
```

2.4 Mitigation Strategy

- Use parameterized queries/prepared statements or a vetted ORM for every database interaction — never build SQL via string concatenation.
- Apply strict, allow-list-based server-side input validation for all fields, complemented by context-aware output encoding (HTML, JS, URL contexts) to prevent XSS.
- Deploy a Content Security Policy to restrict script execution sources as a defense-in-depth layer against XSS.
- Harden session handling: Secure, HttpOnly, SameSite cookies; session ID regeneration on login/privilege change; short idle timeouts; server-side session invalidation on logout.
- Replace verbose database error messages with generic user-facing errors, while logging full details server-side for triage.
- Add a WAF and run regular SAST/DAST scans in the CI/CD pipeline to catch regressions before deployment.

Question 3 — Evil Twin Attack on Corporate WPA2-PSK Wi-Fi (20 Marks)

3.1 How the Evil Twin Attack Was Executed

An Evil Twin attack exploits the fact that Wi-Fi clients identify networks by SSID name alone and will automatically associate with whichever access point advertising that SSID has the strongest signal — there is no built-in way for a client to verify that an AP is the legitimate one. The attacker set up a rogue access point broadcasting the same SSID as the corporate network (and often a spoofed or near-identical BSSID). To accelerate migration of victims onto the rogue AP, the attacker likely transmitted deauthentication frames at the legitimate AP's connected clients, forcing them to disconnect and automatically reconnect — this time to the stronger, attacker-controlled signal. Once employees associated with the rogue AP, the attacker could present a fake captive/login portal to harvest WPA2-PSK or corporate credentials directly, and/or position themselves as a man-in-the-middle for all subsequent traffic, using tools to intercept unencrypted sessions, downgrade HTTPS where possible, and capture login prompts and browsing data.

3.2 Detection and Analysis with Kali Linux Tools

- **Aircrack-ng suite:** airodump-ng passively lists all visible APs and their BSSIDs/channels; a duplicate SSID broadcasting from an unfamiliar or unauthorized BSSID, especially on a different channel, is a strong Evil Twin indicator. airodump-ng also reveals abnormal deauthentication frame counts, which corroborates a forced-reconnection attack.
- **Wireshark:** Once traffic is captured (via a wireless adapter in monitor mode), Wireshark can filter on 802.11 management frames to inspect deauth/disassoc reason codes and sender MAC addresses, and can be used to examine captured session traffic for signs of credential harvesting or SSL-stripping (e.g., unexpected downgrades from HTTPS to HTTP).
- **wifite / Kismet:** Automated wireless auditing tools can flag rogue APs and clients that roam between the legitimate and rogue BSSIDs, speeding up incident triage.

3.3 Weaknesses in the Existing Configuration

- WPA2-PSK uses a single shared passphrase for every device, so there is no per-user identity and no mechanism for a client to authenticate the network back — enabling impersonation.
- No wireless intrusion detection/prevention system (WIDS/WIPS) was in place to automatically flag duplicate SSIDs or excessive deauthentication traffic.
- Employees had no training or tooling to distinguish the legitimate AP from a rogue clone, and the network apparently lacked certificate-based mutual authentication.
- No network segmentation isolated Wi-Fi clients from sensitive internal systems, increasing the value of a successful MITM position.

Recommended Countermeasures

- Migrate from WPA2-PSK to WPA2/WPA3-Enterprise with 802.1X and a RADIUS server, giving each user unique credentials and mutual (certificate-based) authentication that a rogue AP cannot replicate.
- Deploy a WIDS/WIPS to continuously monitor for duplicate SSIDs, unauthorized BSSIDs, and abnormal deauthentication rates, with automated alerting.
- Enable 802.11w (Management Frame Protection), and adopt WPA3 where hardware allows, to authenticate management frames and prevent deauthentication spoofing.
- Require VPN tunneling for access to sensitive internal resources over Wi-Fi, so even a successful MITM position yields only encrypted traffic.
- Run periodic authorized wireless audits with Kali Linux tools to proactively detect rogue infrastructure.
- Conduct user-awareness training so employees recognize unexpected re-authentication prompts or captive portals as red flags and report them.

```
iwconfig
ip link
iw dev
sudo airmon-ng
sudo airmon-ng start wlan0
sudo airodump-ng wlan0mon
sudo airodump-ng --bssid <LAB-AP-MAC> -c <CHANNEL> -w lab_capture wlan0mon
wireshark lab_capture-01.cap
sudo airmon-ng stop wlan0mon
```

Question 4 — Full Attack Chain: Wireless Compromise to Web Application Exploitation (20 Marks)

4.1 Attack Chain Analysis

This scenario chains a wireless security failure into a web application compromise, illustrating why defense-in-depth must cover both layers simultaneously. Weak Wi-Fi passwords give the attacker a foothold on the internal network; once inside, they treat the employee management system as any internal target, exploiting missing authentication and session controls to escalate privileges.

Phase-by-Phase Breakdown

- **1. Reconnaissance:** The attacker surveys nearby wireless networks, identifying the target SSID, signal strength, and encryption type, and may passively capture the WPA2 handshake of connecting clients.
- **2. Wireless Compromise:** Because the passphrase is weak, an offline dictionary/brute-force attack against the captured handshake succeeds, granting network access.
- **3. Internal Network Scanning:** Once on the network, the attacker performs host and service discovery to locate the employee management web application server.
- **4. Web Application Reconnaissance:** The attacker fingerprints the web application's technology stack and enumerates accessible endpoints, forms, and any hidden admin functionality.
- **5. Exploitation:** Weak or missing authentication (e.g., default/guessable credentials, no account lockout) and poor session management (predictable tokens, no timeout, tokens not invalidated on logout) are exploited to obtain a valid, low-privilege session.
- **6. Privilege Escalation:** From a low-privilege session, the attacker probes for broken access control — such as Insecure Direct Object References (IDOR) or missing server-side role checks — to access other employees' records or elevate to administrative functions, ultimately reaching confidential employee data.

4.2 Kali Linux Tools per Phase

Phase	Kali Linux Tool(s)	Purpose
Reconnaissance	airodump-ng, Kismet	Discover SSIDs, capture WPA2 handshakes
Wireless Compromise	aircrack-ng, hashcat, (reaver for WPS if applicable)	Crack weak PSK offline using captured handshake
Network Scanning	Nmap, Netdiscover	Identify live hosts and the web application server/ports
Web App Recon	WhatWeb, Gobuster/dirb	Fingerprint stack; enumerate endpoints and hidden paths
Exploitation	Burp Suite, Hydra (auth testing)	Test authentication flows, session tokens, and input handling
Privilege Escalation	Burp Suite (parameter tampering), Metasploit	Test IDOR/access-control flaws; escalate to admin functions

4.3 Comprehensive, Defense-in-Depth Security Framework

Wireless Layer

- Enforce strong, complex Wi-Fi passphrases (or better, move to WPA2/3-Enterprise with 802.1X/RADIUS).
- Segment wireless clients from critical internal servers using VLANs and firewall rules; place internal applications behind an internal firewall/reverse proxy rather than exposing them flatly on the same broadcast domain.
- Deploy WIDS/WIPS and periodic wireless audits.

Web Application Layer

- Implement strong authentication (complex password policy, account lockout/rate limiting, and ideally multi-factor authentication) for the employee management system.
- Adopt secure session management: random high-entropy tokens, Secure/HttpOnly/SameSite cookies, session expiry, and invalidation on logout.
- Enforce server-side authorization checks (role-based access control) on every request — never rely on client-side or object-ID-based trust — to close IDOR-style privilege-escalation paths.

Organizational Layer

- Adopt a formal, recurring penetration-testing and vulnerability-management program covering both wireless and application layers.
- Centralize logging/SIEM across wireless controllers and the application server to correlate a wireless-layer anomaly with subsequent application-layer activity.
- Provide security awareness training and enforce least-privilege access for all employee accounts.

```
iwconfig
sudo airmon-ng
sudo airmon-ng start wlan0
sudo airodump-ng wlan0mon
ip a
ip route
nmap -sn <lab-subnet>/24
nmap <target-ip>
nmap -sV <target-ip>
nmap -sC -sV <target-ip>
nmap -p- <target-ip>
whatweb http://<target-ip>
nikto -h http://<target-ip>
gobuster dir -u http://<target-ip> -w /usr/share/wordlists/dirb/common.txt
burpsuite
nmap --script vuln <target-ip>
sqlmap -u "http://<target>/page?id=1" --batch
nmap -sV <target-ip> -oN nmap_results.txt
```


Question 5 — Deauthentication Flood Causing Wi-Fi Denial of Service (20 Marks)

5.1 How Deauthentication Attacks Work

The IEEE 802.11 standard (prior to widespread 802.11w/WPA3 adoption) treats management frames — including deauthentication and disassociation frames — as unauthenticated and unencrypted. Any device within radio range can forge a deauthentication frame that appears to originate from a legitimate access point or client, because there is no cryptographic signature binding the frame to its real sender. An attacker first identifies target BSSIDs and connected client MAC addresses using passive monitoring, then repeatedly transmits spoofed deauthentication frames addressed to those clients (or as a broadcast to an entire BSS). Each spoofed frame forces the targeted device to drop its association and attempt to reconnect; if the flood is continuous, devices spend all their time disconnecting and re-associating rather than passing traffic, producing a persistent denial of service across the affected access points during peak hours.

5.2 Simulating and Analyzing the Attack in a Controlled Kali Linux Environment

- airodump-ng: place a wireless adapter in monitor mode and passively enumerate APs, channels, and connected client MAC addresses in the authorized test lab.
- aireplay-ng --deauth (or the more configurable mdk4 deauthentication mode): generate a controlled volume of spoofed deauthentication frames against a lab AP/clients to reproduce and study the effect in isolation, strictly on infrastructure owned or explicitly authorized for testing.
- Wireshark: capture the resulting traffic and filter on 802.11 management-frame subtypes to examine deauth reason codes, source/destination addressing, and the frequency/timing pattern that distinguishes an attack from normal disconnects.
- Correlate the captured pattern with AP-side logs/WIDS alerts to build a signature that can be used for future automated detection.

5.3 Impact on Network Availability and Productivity

- Direct denial of service: legitimate users are repeatedly disconnected and cannot maintain a stable connection during the attack window.
- Productivity loss: coursework, research, and administrative activity dependent on network access (submissions, video lectures, cloud services) are disrupted, particularly damaging during peak/exam periods.
- Secondary risk: forced reconnections are also the mechanism an Evil Twin attacker uses (see Question 3), so unmitigated deauthentication capability is a stepping-stone to more serious credential-theft or MITM attacks, not just an availability issue.

5.4 Mitigation Strategies

- Enable 802.11w (Management Frame Protection) on all access points so deauthentication/disassociation frames must be cryptographically authenticated, neutralizing spoofed frames.
- Migrate to WPA3, which mandates Protected Management Frames as part of the standard.
- Deploy a Wireless Intrusion Detection/Prevention System (WIDS/WIPS) to detect abnormal deauthentication frame volumes in real time and alert network administrators.
- Increase AP density/channel planning and enable band steering so the network is more resilient to localized jamming/flooding attempts.
- Establish an incident-response runbook for wireless DoS events, including physical location of rogue transmitters using signal-strength triangulation.
- Maintain firmware/driver updates on access points and client devices to ensure 802.11w and related protections are actually supported and enabled.

```
iwconfig
iw dev
sudo airmon-ng start wlan0
sudo airodump-ng wlan0mon
sudo airodump-ng --bssid <LAB-AP-MAC> -c <CHANNEL> -w deauth_lab wlan0mon
wireshark deauth_lab-01.cap
sudo airmon-ng stop wlan0mon
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity difficult to understand